

Deep Learning – HW2

Amit Rubinshtein – 324300722 – amit.ru@campus.technion.ac.il

Nadav Offir – 213786197 - Nadav.offir@campus.technion.ac.il

Question 1

$$\begin{aligned}\bar{R} &= E \left\| (H_\mu^{-1}H - I)w_{true} + H_\mu^{-1}X^T\epsilon \right\|^2 \\&= E \left\| (H_\mu^{-1}H - I)w_{true} \right\|^2 + E \left\| H_\mu^{-1}X^T\epsilon \right\|^2 \\&= E \left[w_{true}^T (H_\mu^{-1}H - I) (H_\mu^{-1}H - I) w_{true} \right] + E \left\| \epsilon^T X H_\mu^{-1} H_\mu^{-1} X^T \epsilon \right\| \\&= \sum_{i,j=1}^d E[w_{i,true}w_{j,true}] (H_\mu^{-1}H - I)_i^T (H_\mu^{-1}H - I)_j + \sum_{i,j=1}^N E[\epsilon_i\epsilon_j] (XH_\mu^{-1})_i (H_\mu^{-1}X^T)_j \\&= \sum_{i,j=1}^d \frac{\sigma_w^2}{d} \delta_{ij} (H_\mu^{-1}H - I)_i^T (H_\mu^{-1}H - I)_j + \sum_{i,j=1}^N \sigma_\epsilon^2 \delta_{ij} (XH_\mu^{-1})_i (H_\mu^{-1}X^T)_j \\&= \frac{\sigma_w^2}{d} \sum_{i=1}^d (H_\mu^{-1}H - I)_i^T (H_\mu^{-1}H - I)_i + \sum_{i=1}^N \sigma_\epsilon^2 (XH_\mu^{-1})_i (H_\mu^{-1}X^T)_i \\&= \frac{\sigma_w^2}{d} \text{Tr} \left[(H_\mu^{-1}H - I)^2 \right] + \sigma_\epsilon^2 \text{Tr} [XH_\mu^{-2}X^T] \\&= \frac{\sigma_w^2}{d} \text{Tr} \left[(H_\mu^{-1}H - I)^2 \right] + \sigma_\epsilon^2 \text{Tr} [X^T X H_\mu^{-2}] \\&= \frac{\sigma_w^2}{d} \text{Tr} [(H + \mu I)^{-1}H - I]^2 + \sigma_\epsilon^2 \text{Tr} [H(H + \mu I)^{-2}] \\&= \frac{\sigma_w^2}{d} \text{Tr} [(U\Lambda U^T + \mu I)^{-1}U\Lambda U^T - I]^2 + \sigma_\epsilon^2 \text{Tr} [U\Lambda U^T (U\Lambda U^T + \mu I)^{-2}] \\&= \frac{\sigma_w^2}{d} \text{Tr} [(U(\Lambda + \mu I)^{-1}\Lambda U^T - I)^2] + \sigma_\epsilon^2 \text{Tr} [U\Lambda(\Lambda + \mu I)^{-2}U^T] \\&= \frac{\sigma_w^2}{d} \text{Tr} [(\Lambda + \mu I)^{-1}\Lambda - I]^2 + \sigma_\epsilon^2 \text{Tr} [\Lambda(\Lambda + \mu I)^{-2}] \\&= \frac{\sigma_w^2}{d} \sum_{i=1}^d \left(\frac{\lambda_i}{\lambda_i + \mu} - 1 \right)^2 + \sigma_\epsilon^2 \sum_{i=1}^d \frac{\lambda_i}{(\lambda_i + \mu)^2} \\&= \sum_{i=1}^d \frac{\frac{\sigma_w^2 \mu^2}{d} + \sigma_\epsilon^2 \lambda_i}{(\lambda_i + \mu)^2}\end{aligned}$$

Question 2

1. With $\mathbf{W}_1$ frozen and $\mathbf{v}^{(n)} = \mathbf{W}_1 \mathbf{x}^{(n)}$ the network becomes:

$$f(\mathbf{x}^{(n)}) = \mathbf{w}_2^T \mathbf{v}^{(n)}$$

with $\mu = 0$:

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^N (\mathbf{y}^{(n)} - \mathbf{w}_2^T \mathbf{v}^{(n)})$$

we will re-write the problem with:

$$\mathbf{V} \in \mathbb{R}^{N \times k}, V_{n,i} = v_i^{(n)}$$

then the network output will be:

$$\hat{\mathbf{y}} = \mathbf{V} \mathbf{w}_2$$

and:

$$\mathcal{L} = \frac{1}{N} \|\mathbf{y} - \mathbf{V} \mathbf{w}_2\|_2^2$$

we will reach zero training error when $\mathbf{y} = \mathbf{V} \mathbf{w}_2$. Since $\mathbf{v}^{(n)}$ span an N -dimensional space then $\text{Rank}(\mathbf{V}) = N$. Therefore for any $\mathbf{y} \in \mathbb{R}^N$ exist $\mathbf{w}_2 \in \mathbb{R}^k$ such that $\mathbf{V} \mathbf{w}_2 = \mathbf{y}$.

In addition, because $\mathcal{L} = \frac{1}{N} \|\mathbf{y} - \mathbf{V} \mathbf{w}_2\|_2^2$ is a convex function, gradient flow will reach its minimum which is when $\mathbf{V} \mathbf{w}_2 = \mathbf{y}$.

2. No, $\mathbf{w}_2$ cannot leave subspace spanned by $[\mathbf{v}^{(n)}]_{n=1}^N$. Let's look at the gradient step:

$$\frac{d\mathbf{w}_2}{dt} = -\frac{d\mathcal{L}}{d\mathbf{w}_2} = -\frac{2}{N} \mathbf{V}^T (\mathbf{V} \mathbf{w}_2 - \mathbf{y}) \in \text{span}\{\mathbf{v}^{(1)}, \dots, \mathbf{v}^{(N)}\}$$

let's denote $S = \text{span}\{\mathbf{v}^{(1)}, \dots, \mathbf{v}^{(N)}\}$.

Now:

$$\mathbf{w}_2(0) = \mathbf{0} \in S$$

because $\mathbf{0}$ is in every subspace. Using induction, we can see $\mathbf{w}_2(t) \in S$.

3. Using Lagrange multipliers:

$$\mathcal{L}(\mathbf{w}_2, \boldsymbol{\lambda}) = \|\mathbf{w}_2\|^2 + \boldsymbol{\lambda}^T (\mathbf{V} \mathbf{w}_2 - \mathbf{y})$$

the optimality conditions are:

$$\frac{d\mathcal{L}}{d\mathbf{w}_2} = 2\mathbf{w}_2 + \boldsymbol{\lambda}^T \mathbf{V} = \mathbf{0}$$

$$\mathbf{V} \mathbf{w}_2 = \mathbf{y}$$

we will get:

$$\mathbf{w}_2 = -\frac{1}{2} \mathbf{V}^T \boldsymbol{\lambda}$$

$$-\frac{1}{2} \mathbf{V} \mathbf{V}^T \boldsymbol{\lambda} = \mathbf{y}$$

$$\boldsymbol{\lambda} = -2(\mathbf{V} \mathbf{V}^T)^{-1} \mathbf{y}$$

$$\mathbf{w}_2^* = \mathbf{V}^T (\mathbf{V}\mathbf{V}^T)^{-1} \mathbf{y}$$

Now, let's look at GD step:

$$\mathcal{L}(\mathbf{w}_2) = \frac{1}{N} \|\mathbf{y} - \mathbf{V}\mathbf{w}_2\|_2^2$$

we have shown in the previous section that $\mathbf{w}_2 \in S$, so we can write $\mathbf{w}_2 = \mathbf{V}^T \boldsymbol{\alpha}$:

$$\mathcal{L}(\boldsymbol{\alpha}) = \frac{1}{N} |\mathbf{V}\mathbf{V}^T \boldsymbol{\alpha} - \mathbf{y}|^2$$

which is strongly convex in $\boldsymbol{\alpha}$. Therefore:

$$\boldsymbol{\alpha} = (\mathbf{V}\mathbf{V}^T)^{-1} \mathbf{y}$$

and:

$$\mathbf{w}_2 = \mathbf{V}^T (\mathbf{V}\mathbf{V}^T)^{-1} \mathbf{y}$$

4. We will converge to a solution that will not get us zero training error but will make $|\mathbf{w}_2|^2$ smaller. The bigger μ the larger the error will be and the smaller $|\mathbf{w}_2|^2$ will be.

Question 3

1. 200-300 wise Double Descent
2. 10 wise Double Descent
3. High-capacity models can exhibit a 'double descent' trajectory: the test error initially falls, spikes near the interpolation threshold, and then drops once more. This differs from 'medium' models those just capable of reaching zero training error which follow the traditional U-shaped curve regarding training time, implying early stopping is optimal. Conversely, models that are too small to fit the data remain in the 'under-parameterized' zone, where extending training time results in a continuous, monotonic reduction in test error.

Question 4

1. If u 's distribution is symmetric and $\phi(z) = \max(0, z)$, then we can rewrite the expectation as:

$$\begin{aligned}\mathbb{E}[\phi^2(u)] &= p(u > 0)\mathbb{E}_{u>0}[\phi^2(u)] + p(u < 0)\mathbb{E}_{u<0}[\phi^2(u)] = \frac{1}{2}\mathbb{E}_{u>0}[u^2] \\ &= \frac{1}{2}\mathbb{E}[u^2]\end{aligned}$$

we can do the last equality because u is symmetric. Under the assumption that is used in the calculation of σ_l :

$$\text{Var}(u_{l-1}) = \mathbb{E}[u_{l-1}^2] = 1$$

we get:

$$\sigma_l = \frac{1}{\sqrt{\sum_j \mathbb{E}[\phi^2(u_{l-1}[j])]}} = \frac{1}{\sqrt{\frac{1}{2}\sum_j \mathbb{E}[u_{l-1}^2[j]]}} = \sqrt{\frac{2}{d_{l-1}}}$$

2. Using the central limit for large width:

$$\sigma_l = \frac{1}{\sqrt{d_{l-1} \mathbb{E}_{z \sim N(0,1)}[\phi^2(z)]}}$$

because $N(0,1)$ is a symmetric distribution using the previous part we can say:

$$\mathbb{E}_{z \sim N(0,1)}[\phi^2(z)] = \frac{1}{2} \mathbb{E}_{z \sim N(0,1)}[z^2] = \frac{1}{2}$$

therefore:

$$\sigma_l = \sqrt{\frac{2}{d_{l-1}}}$$

Question 5

First direction: Assume $W_{i,j} = W_{\tau(i),\tau(j)}$ We evaluate the i-th component of $f_w(\tau \cdot x)$:

$$[f_w(\tau \cdot x)]_i = \phi \left(\sum_j W_{i,j} x_{\tau^{-1}(j)} \right)$$

Change variables $k = \tau^{-1}(j) \Rightarrow j = \tau(k)$:

$$= \phi \left(\sum_k W_{i,\tau(k)} x_k \right)$$

By the weight symmetry assumption ($W_{i,\tau(k)} = W_{\tau^{-1}(i),k}$):

$$= \phi \left(\sum_k W_{\tau^{-1}(i),k} x_k \right) = \phi([Wx]_{\tau^{-1}(i)}) = [\tau \cdot f_w(x)]_i$$

Thus, the function is equivariant.

Second direction: Assume $f_w(\tau \cdot x) = \tau \cdot f_w(x)$.

$$\phi(W(\tau \cdot x)) = \tau \cdot \phi(Wx)$$

Because ϕ is injective, we can apply ϕ^{-1} to both sides, reducing the problem to the linear case:

$$W(\tau \cdot x) = \tau \cdot (Wx)$$

Expanding both sides for an arbitrary index i:

$$\sum_k W_{i,\tau(k)} x_k = \sum_k W_{\tau^{-1}(i),k} x_k$$

Since this holds for all x, the coefficients must be identical:

$$W_{i,\tau(k)} = W_{\tau^{-1}(i),k}$$

Let $i = \tau(a)$ and $k=b$. This yields the required condition:

$$W_{\tau(a),\tau(b)} = W_{a,b}$$

Question 6

1. We will use the the following equation to calculate the output width of a convolution layer:

$$W_{out} = \left\lfloor \frac{W_{in} - F + 2P}{S} \right\rfloor + 1$$

and max pooling of 2 by 2 with a stride of 2 will half the width. The number of the parameters for the convolution layer is calculated by

$$number\ of\ parameters = W_{kernel}^2 \cdot C_{in} \cdot C_{out} + C_{out}$$

where W_{kernel} is the width of the kernel, C_{in} is the number of channel in the input and C_{output} is number of channel in the output. For a fully connected layer:

$$number\ of\ parameters = N_{in} \cdot N_{out} + N_{out}$$

Layer	Output Dimension	Number of Parameters (Weights)
INPUT	224x224x3	0
CONV3-64	224x224x64	1,792
ReLU	224x224x64	0
POOL2	112x112x64	0
CONV3-128	112x112x128	73,856
ReLU	112x112x128	0
POOL2	56x56x128	0
CONV3-256	56x56x256	295,168
ReLU	56x56x256	0
CONV3-256	56x56x256	590,080
ReLU	56x56x256	0
POOL2	28x28x256	0
CONV3-512	28x28x512	1,180,160
ReLU	28x28x512	0
CONV3-512	28x28x512	2,359,808
ReLU	28x28x512	0
POOL2	14x14x512	0
CONV3-512	14x14x512	2,359,808
ReLU	14x14x512	0
CONV3-512	14x14x512	2,359,808
ReLU	14x14x512	0
POOL2	7x7x512	0
FC-4096	1x4096	102,764,544
FC-4096	1x4096	16,781,312
FC-1000	1x1000	4,097,000
SOFTMAX	1x1000	0

2. The number of parameters in the convolutional layers are: 9,220,480. And in the fully connected: 123,642,856. The total number: 132,863,336.

3. $\frac{123,642,856}{132,863,336} \cdot 100\% = 93.06\%$